

Z1 Controller Implementation Review

Machine control, CAM algorithms and path design, and the JavaScript CAM language

REVIEW DATE 2026-08-12	REVIEW TARGET task/cnc-control-dropcut e82bed1e5a00f38f...
SUPPLIED ARCHIVE dropcut-studio.zip SHA-256 61de82451c552edd...	DISPOSITION NO-GO for production cutting

Static review, executable probes, build analysis, design-document comparison, and upstream provenance verification. No physical machine was operated.

Contents

Executive assessment

- Verdict
- Release recommendation
- Most important actions

Scope, evidence, and limitations

- Material reviewed
- Provenance
- Methods
- What was not established

Architecture assessment

- What is structurally strong
- The principal systems gap

Machine control design

- Summary
- Finding matrix
- MC-01 - Generic command classification is fail-open
- MC-02 - Admission and execution are not atomic
- MC-03 - Timeout leaves an ambiguous session
- MC-04 - Protocol replacement is racy and weakly authenticated
- MC-05 - Short writes are treated as success
- MC-06 - Safety-relevant inbound data shares a lossy queue
- MC-07 - `play` can silently fail and still complete successfully
- MC-08 - `CycleStart` bypasses the state-enabling path
- MC-09 through MC-11 - Additional control findings

CAM algorithms and path design

- Summary
- Finding matrix
- CAM-01 - The certificate asserts a gouge check that does not exist
- CAM-02 - Contour chaining collides inside the machine envelope
- CAM-03 - Work offsets and machine travel are modeled in incompatible frames
- CAM-04 - The promised safety epilogue is not emitted
- CAM-05 - Certificate branding is not a trust boundary
- CAM-06 - Probe semantics are emitted incorrectly
- CAM-07 - A missing traverse is converted into a cutting move
- CAM-08 - Raw text invalidates modal compression
- CAM-09 and CAM-10 - Identity and initial state are assumed
- CAM-11 - Linking and entry use an incomplete clearance model
- CAM-12 - Adaptive refinement does not prove its stated bound
- CAM-13 - The constant-scallop name overstates current behavior
- CAM-14 and CAM-15 - Path semantics and operation domains need stronger contracts
- CAM-16 through CAM-23 - Additional algorithm findings

JavaScript API and CAM language design

- Summary
- Finding matrix
- JS-01 - This is not a security boundary and has no timeout
- JS-02 - Unit tags do not enforce numeric validity or machining domains
- JS-03 - Tool definitions are mutable after registration
- JS-04 - Async callbacks violate lexical scope semantics
- JS-05 - Spindle scope documentation and execution disagree
- JS-06 - `workOffset` is a dead and unsafe language feature
- JS-07 - Reproducibility is not a language invariant
- JS-08 - Separate the authoring frontend from the stable CAM language
- JS-09 - The handbook is ahead of the implementation

Cross-system trust, deployment, and operations

- The missing boundary is a deployment bundle
- SYS-01 - Job start does not prove the job actually started
- SYS-02 - Controller session state is not a safety state machine
- SYS-03 - The web UI is locally hardened but remote mode needs a secure transport story
- Reproducibility and test status
- Licensing and process boundaries

Recommended target architecture

Remediation plan

- P0 - Required before cutting material
- P1 - Required before production or unattended use
- P2 - Quality and capability maturation

Acceptance gates

- Gate A - Script host
- Gate B - Language and plan validation
- Gate C - Geometry and algorithms
- Gate D - Compiler and postprocessor
- Gate E - Certificate
- Gate F - Controller
- Gate G - Physical bring-up

Overall conclusion

Appendix A - Executable probe results

Appendix B - Build, test, and provenance record

- Review-environment commands and failure modes

Appendix C - Severity model

Appendix D - Review checklist by subsystem

- Machine control
- CAM and postprocessing
- JavaScript language

Appendix E - Minimum release checklist

Executive assessment

Verdict

The implementation is not ready for production cutting, unattended operation, or remote operation over an untrusted network. It is suitable as a research prototype for simulation, code review, controlled dry runs, and supervised air cuts. Cutting trials should remain limited to sacrificial stock and a physically attended operator with immediate access to the machine's hardware emergency stop until the P0 gates in this report are closed.

The architecture has several strong ideas: a non-modal semantic CAM pipeline, typed machine-motion operations, a dedicated risk model, fresh preflight reads, a dead-man jog design, capability-driven postprocessing, and an explicit certificate that is intended to state what was and was not verified. Those are the right abstractions.

The current implementation fails at the boundaries where those abstractions are supposed to become safety guarantees. Four defects are release-blocking:

1. **The certificate can report gouge verification without performing a gouge comparison against a target surface.** A stock-removal and rapid-crash pass is unconditionally promoted into both `gouge` and `rapidCrash` certificate rows.
2. **The JavaScript “sandbox” is same-realm `new Function` execution.** Constructor-chain escape reaches the host global object, and `timeoutMs` is accepted but not enforced. In the Studio this runs on the UI thread.
3. **The generic controller command classifier is fail-open.** It classifies only the first token and returns `ClassRead` for unknown verbs. Compound blocks and embedded newlines can place motion behind an allowed first token.
4. **Contour endpoint keys wrap every 67.108864 mm.** That distance lies well inside the Z1's 200 mm XY envelope, so unrelated contour segments can merge and even be marked closed.

The next tier of high-severity issues is also material: work offsets are accepted but not emitted; travel is “verified” in a simplified work frame rather than the machine frame; the validator promises an appended `M5` that the emitter does not append; probing is emitted as the wrong endpoint in absolute mode; raw G-code can invalidate modal compression; a missing planner traverse is silently replaced by a cutting move; machine-profile identity is not checked; toolpath entry and linking do not track evolving stock; and the controller has timeout, partial-write, protocol-race, and message-loss failure modes.

Release recommendation

Use	Recommendation	Conditions
Source development and simulation	GO	Treat all certificate output as advisory until CAM-01 is fixed.
Parser/postprocessor regression work	GO WITH LIMITS	Disable raw escapes; inspect uncompressed and compressed output; require deterministic test fixtures.
Dry run or air cut on a local machine	CONDITIONAL	Loopback control only, physical E-stop, independent G-code review, machine homed and WCS verified manually, no unattended job start.

Use	Recommendation	Conditions
Cutting sacrificial foam/wax	CONDITIONAL AFTER P0	Close the four critical defects plus WCS/footer/numeric validation gates; use conservative feeds and stock clearance.
Production material, unattended jobs, or remote shop operation	NO-GO	Requires P0 and P1 completion, hardware validation, artifact binding, and an operational safety case.

Most important actions

Priority	Required action	Why it blocks release
P0-1	Split sampled evidence by check and implement a real target-surface gouge comparison.	Current certificate can state a fact that was never checked.
P0-2	Run scripts in a hard-terminable isolated worker or subprocess; remove same-realm execution.	A script can escape capabilities, access host resources, and hang the Studio.
P0-3	Replace free-text command classification with a fail-closed grammar that examines every line and word.	The generic “read-only” path can transmit motion.
P0-4	Replace modulo-packed contour keys with collision-free tuple keys or topology IDs.	Geometry can be silently corrupted inside the normal work envelope.
P0-5	Make frame transforms and WCS selection explicit, validated, and emitted.	Travel certification and actual machine coordinates are currently disconnected.
P0-6	Enforce a canonical safety epilogue and modal barriers; fix probe and cut-start behavior.	The postprocessor can leave the spindle on, emit a wrong probe, or change intended motion.
P0-7	Apply complete finite/domain validation and immutability to the JS API.	Invalid dimensions can enter algorithms, mutate after registration, or trigger nontermination.
P0-8	Bind the <code>.nc</code> file to a deployment manifest and certificate consumed by <code>z1ctl</code> .	The controller can play any uploaded file, independently of the CAM evidence.

Scope, evidence, and limitations

Material reviewed

The supplied archive contains a TypeScript CAM monorepo and a nested Go controller/CLI:

- 87 TypeScript/TSX source and test files across `packages/` and `apps/`, approximately 15,174 lines.
- 50 Go files under `makera-z1-cli/`, approximately 11,973 lines.
- 67 Markdown design, protocol, diary, and review documents, approximately 22,425 lines.
- Original prototypes and design notes under `original/`.
- Pinned firmware and controller evidence under the `MZ1-*` research directories.

The audit covered the end-to-end flows rather than only named modules:

```
JavaScript source
-> script host and capability API
-> ManufacturingPlan
-> strategies, path ordering, linking, entry and refinement
-> CanonicalProgram
-> machine lowering and exact validation
-> RS-274 / Makera emission and modal compression
-> sampled stock checks and certificate re-issuance
-> .nc artifact

.nc artifact
-> zlctl upload and digest check
-> fresh machine preflight
-> play command
-> progress/alarm supervision
-> Makera/Carvera firmware
```

Provenance

The archive's `.git` file is not portable. It contains an absolute worktree pointer:

```
gitdir: /home/manuel/code/wesen/dropcut-studio/.git/worktrees/dropcut-studio
```

Repository metadata was therefore reconstructed from GitHub and the source snapshot. The supplied source corresponds to the public `wesen/dropcut-studio` branch `task/cnc-control-dropcut`, with review anchor commit:

```
e82bed1e5a00f38f4441e6ea13e1265edc775928
```

The controller research pins stock firmware evidence to:

```
MakeraInc/CarveraFirmware
1683b6fb5c7ec1d341c476c6fdb2a22f7a26220e
```

and community firmware evidence to:

Fresh `git clone` operations for upstream repositories could not complete in the execution environment because outbound Git DNS was unavailable. Repository and commit metadata were verified through GitHub, and the review used the complete supplied snapshot plus its pinned source excerpts. This limitation affects reproducibility of the review environment, not the local source findings.

Methods

The review used four evidence classes:

1. **Static source inspection.** Control flow, concurrency, state ownership, safety policy, coordinate frames, numerical assumptions, postprocessor modality, and public API contracts were traced across package boundaries.
2. **Executable probes.** Core TypeScript packages were compiled and imported, then 15 focused probes exercised sandbox escape, timeout behavior, unit validation, mutation, async scope behavior, spindle transitions, work-offset emission, footer generation, probe emission, cut-start gaps, raw modal compression, certificate re-issuance, profile identity, certificate completeness, and contour-key collision.
3. **Build and test analysis.** Test inventory, toolchain declarations, workspace dependencies, and build reproducibility were checked. A custom core TypeScript typecheck completed with no errors for the dependency-free core subset.
4. **Design-to-implementation comparison.** The implementation was checked against `CAM-001`, `MZ1-001`, `MZ1-002`, `MZ1-003`, and the original semantic-CAM and Kleisli-composition design notes.

What was not established

- No physical Z1 was connected or moved.
- No spindle was started and no material was cut.
- The full pnpm/Vitest suite could not be run because package-manager dependencies were not available offline.
- The Go suite could not be executed because the module requires Go 1.26.1, the environment has Go 1.23.2, the declared `../../glazed` sibling checkout is absent, and the archive has no `go.sum` or `go.work`.
- This is an engineering implementation review, not a functional-safety certification, machinery risk assessment, or legal determination of regulatory compliance.

Architecture assessment

What is structurally strong

The project rejects G-code as the semantic source of truth and keeps modality until the final compression pass. That is a sound design. The chain from `ManufacturingPlan` through canonical commands, machine lowering, validation, emission, and analysis is inspectable and testable. Provenance and motion timelines are carried through the pipeline, and machine behavior is represented as profile data rather than scattered target-specific branches.

The Go controller similarly contains several good safety decisions:

- Typed `MotionOp` constructors keep ordinary callers away from raw command strings.
- Risk classes distinguish stops, accessories, data mutation, state-enabling actions, and motion.
- Stop paths are intentionally not blocked by machine-state preflight.
- Motion requests do not retry after ambiguous failure.
- Continuous jog relies on firmware dead-man behavior and the browser-held keepalive, rather than a server timer that could keep an abandoned jog alive.
- The web server binds to loopback by default and adds Host, same-origin, fetch-metadata, and token checks to mutating routes.
- File transfer has a single reader and an explicit transfer-mode ownership model.

These are not cosmetic strengths. They show that the implementation is organized around hazards rather than only features.

The principal systems gap

The CAM and controller are not joined by a trust contract. CAM emits a text file and an in-memory certificate. `z1ctl` accepts a local file, checks transfer integrity with MD5/size, and plays it after machine-state preflight. It does not know:

- which source script produced the file;
- which machine profile and profile revision were used;
- which WCS/setup was assumed;
- which tool table, stickout, holder, fixture, or stock model was certified;
- whether simulation ran;
- which certificate rows were exact, sampled, skipped, or unverifiable;
- whether the file has been modified since certification.

The most important architectural change is therefore not another UI check. It is a hash-bound deployment manifest that closes the gap between planning evidence and machine execution.

Machine control design

Summary

The machine-control layer is better structured than a typical ad hoc CNC client. Its typed motion requests, no-retry rule, fresh preflight, dead-man jog, and loopback-first web posture should be preserved. The main problems are at the generic-command boundary and in session-level concurrency/recovery. The implementation serializes individual command exchanges, but it does not provide a transaction that spans preflight and all commands of an authorized operation. It also assumes a timed-out session can immediately resume normal command exchange, which is unsafe for a sentinel protocol.

Finding matrix

ID	Severity	Finding	Primary evidence
MC-01	CRITICAL	Generic command classification is fail-open and examines only the first token.	<code>makera-z1-cli/pkg/makera/safety.go:152-233</code> ; <code>makera-z1-cli/pkg/makera/protocol.go:76-82</code>
MC-02	HIGH	Preflight and execution are not one atomic operation; commands can interleave after admission.	<code>makera-z1-cli/pkg/makera/client.go:99-104</code> ; <code>makera-z1-cli/pkg/makera/preflight.go:88-170</code> ; <code>makera-z1-cli/pkg/makera/motion.go:807-856</code>
MC-03	HIGH	Timeout/cancellation can leave a late sentinel or reply in the stream and desynchronize the next command.	<code>makera-z1-cli/pkg/makera/client.go:304-358</code>
MC-04	HIGH	Protocol replacement and decoder use are unsynchronized; arbitrary reply text can trigger a dialect switch.	<code>makera-z1-cli/pkg/makera/client.go:180-225</code> ; <code>protocol.go</code>
MC-05	HIGH	Transport writes ignore short-write counts.	<code>makera-z1-cli/pkg/makera/client.go:275-282</code>
MC-06	HIGH	Inbound messages can be dropped or drained, including alarms and state transitions.	<code>makera-z1-cli/pkg/makera/client.go:245-272</code> ; <code>makera-z1-cli/pkg/makera/client.go:292-302</code>
MC-07	HIGH	An unhomed stock-firmware <code>play</code> may silently no-op and be reported as completion.	<code>makera-z1-cli/pkg/makera/preflight.go:145-158</code> ; <code>makera-z1-cli/cmd/z1ctl/cmds/job.go:340-378</code>
MC-08	HIGH	Exported realtime <code>CycleStart()</code> re-enables motion without preflight or an admission token.	<code>makera-z1-cli/pkg/makera/jobctl.go:49-70</code>
MC-09	MEDIUM	Remote web mode protects mutations with a bearer token but sends it and commands over plaintext HTTP.	<code>makera-z1-cli/cmd/z1ctl/cmds/serve.go:100-147</code> ; <code>makera-z1-cli/pkg/webui/motion.go:43-82</code>

ID	Severity	Finding	Primary evidence
MC-10	MEDIUM	The framed decoder documents a lossy false-header case; recovery is not integrated with session quarantine.	<code>makera-z1-cli/pkg/makera/frame.go</code> ; <code>makera-z1-cli/pkg/makera/frame_test.go:129-145</code>
MC-11	MEDIUM	Controller documentation and build metadata are stale or incomplete.	<code>makera-z1-cli/README.md:33-68</code> ; <code>go.mod</code> ; missing <code>go.sum</code> / <code>go.work</code>

MC-01 - Generic command classification is fail-open

Evidence. `Classify` trims and splits the command, considers `parts[0]`, and returns `ClassRead` for any unknown verb. For G/M/T-like input it classifies only the leading code. `MakeraProtocol.EncodeCommand` removes trailing CR/LF but preserves embedded separators and newlines. The tests cover isolated commands, not compound blocks or multiline payloads.

Representative bypass classes are:

```
M5 G0 X10      # first M code is a stop; later G0 is not examined
M957 G0 X10    # read-only M-code exception masks later words
version\nG0 X10 # unknown/read verb masks a second command line
future-dangerous # unknown firmware verb defaults to read
```

The exact grammar accepted by firmware may differ by dialect, but the safety property must not depend on firmware rejecting the second word. A generic path described as read-only must reject any input it cannot prove read-only.

Impact. A caller with access only to `Client.Command` can potentially transmit motion or state-changing content. This defeats the most important control-layer boundary.

Required change. Replace heuristic risk classification with a parser that:

- rejects CR, LF, NUL, comments/separators, and multiple logical lines on the generic path;
- tokenizes every word in a G-code block and computes the maximum risk across all words;
- recognizes an explicit allowlist of read-only shell commands and their argument grammars;
- treats unknown verbs, unknown codes, and malformed input as denied, not read-only;
- has fuzz/property tests for concatenation, whitespace, casing, comments, line breaks, and every pair of allowed/disallowed tokens.

The safest design is to remove free-text execution from ordinary API consumers and expose typed read commands only. Keep raw execution behind a separately named diagnostic interface that is disabled by default and never used by the web UI.

MC-02 - Admission and execution are not atomic

`cmdMu` protects one drain-write-collect exchange. `Preflight` performs `QueryStatus` and `QueryDiagnose` as separate exchanges. `Motion` then loops over rendered commands and calls `commandUnchecked` once per command. Another goroutine can run between status and diagnose, after preflight, or between two operations. Realtime paths deliberately bypass `cmdMu`.

This is a classic time-of-check/time-of-use problem. The web server reduces exposure by single-flighting HTTP motion and serializing its client callback, but the library and CLI API do not enforce an operation-wide state machine.

Required change. Add a session admission/execution mutex or explicit state machine covering:

```
fresh status + diagnose -> decision -> all commands -> final status
```

Stop-class traffic must remain on a separate priority lane and must not wait for the operation lock. Other reads can either queue behind the operation or consume from a separate telemetry stream with defined consistency semantics.

MC-03 - Timeout leaves an ambiguous session

A command writes the requested command and a fixed `echo EOT` sentinel, then waits up to `CommandTimeout`. On timeout or context cancellation it returns and releases `cmdMu`. The firmware can still deliver the command's output and sentinel later. The next command drains buffered messages and can discard that late output, or a late sentinel can terminate the wrong exchange.

Required change. Treat an exchange timeout as a protocol-session fault. Close and reconnect, or enter a quarantine/resynchronization procedure that proves the old command and sentinel have been consumed before accepting a new operation. If firmware permits it, use a unique per-command nonce rather than a fixed sentinel. Never infer that a timed-out motion command did not execute.

MC-04 - Protocol replacement is racy and weakly authenticated

The read loop calls `ProtocolFromAnnouncement(m.Text)` for every decoded message and directly assigns `c.proto = NewProtocol(announced)`. Other goroutines read and mutate the same protocol object for encoding, framing, decoder state, name, and drop counters without synchronization.

An ordinary reply containing the announcement phrase can also trigger a mid-session dialect switch. That may reset decoder state while bytes from the previous dialect remain in flight.

Required change. Restrict protocol selection to a startup handshake or explicitly framed, trusted announcement state. Make protocol selection immutable for a session where possible. Otherwise perform a synchronized transition that stops writers, drains the old decoder, validates the new handshake, swaps an atomic immutable strategy, and resumes.

MC-05 - Short writes are treated as success

`Client.write` ignores the `n` returned by `Transport.Write`. TCP writes often complete, but the Go `io.Writer` contract permits `n < len(b)` with or without error. Framed realtime and command data can therefore be truncated while the caller proceeds as though it was sent.

Required change. Use a `writeFull` loop for byte transport, or strengthen the `Transport` contract and enforce it. A short write during a motion or realtime command is an ambiguous failure; close the session rather than semantically retrying the command.

MC-06 - Safety-relevant inbound data shares a lossy queue

The read loop never blocks. When the 256-message channel fills, `deliver` drops the oldest or incoming message, preserving only the fixed sentinel. Before each exchange `drain()` discards pending output. That policy is reasonable for debug chatter but not for alarms, cover events, halt reasons, or stop acknowledgements.

Required change. Separate inbound traffic into at least:

- non-dropping or latched safety events;
- command-correlated replies;
- telemetry/status snapshots;
- lossy informational logs.

An alarm should remain observable until acknowledged by the application. Draining a command queue must not clear safety state.

MC-07 - `play` can silently fail and still complete successfully

The implementation correctly documents that stock firmware cannot reliably report the homed flag. It also documents that `play` can silently no-op when unhomed. The preflight homing row is advisory, not fatal. `job run` sends play and then polls status; any state with no active playback is reported as complete.

A job that never entered the active-playing state can therefore be reported as successful completion.

Required change. After `play`, require an observed transition into an active playback state within a short start deadline before monitoring completion. If stock firmware cannot provide that transition reliably, require a fresh operator homing attestation bound to the session and display a distinct “start unverified” state. Never collapse “never started” and “finished” into the same result.

MC-08 - `CycleStart` bypasses the state-enabling path

`Resume(ctx, opts)` performs preflight. `CycleStart()` directly writes the realtime `~` byte. Both restart motion, but only one carries context, admission checks, or an auditable reason.

Required change. Make raw cycle start private to an admitted hold/resume session. Expose an API such as `ReleaseFeedHold(ctx, holdToken, opts)` that proves the client established the corresponding hold and that the machine state still matches the expected session.

MC-09 through MC-11 - Additional control findings

Remote HTTP. The loopback default, Host validation, same-origin checks, `Sec-Fetch-Site`, body limits, and non-loopback token are good. The remaining issue is transport confidentiality and token handling: remote mode prints a URL containing the token and serves plaintext HTTP. On a shared or untrusted LAN, an observer can recover the token and motion traffic. Bind to loopback by default in production; require a TLS reverse proxy, mTLS, or a trusted tunnel for remote access. Do not put bearer tokens in query strings.

Decoder recovery. The false-header loss case is openly documented and tested, which is good. A nonzero drop count should nevertheless mark the session suspect. Motion admission should refuse until reconnect/resync, rather than merely exposing a diagnostic counter.

Documentation/build. The nested README still says motion and job control are not implemented, even though they are. `go.mod` requires Go 1.26.1 and replaces `glazed` with an absent sibling checkout; the archive lacks the referenced `go.work` and a `go.sum`. This prevents an independent reviewer from reproducing the controller test suite from the archive. Pin a released `glazed` version or vendor it, include checksums/workspace metadata, and generate feature/status documentation from executable commands and tests.

CAM algorithms and path design

Summary

The CAM core has a credible decomposition. Strategies propose geometry; the planner orders paths, creates entry and linking motion, and lowers operations to a canonical non-modal program; the machine layer resolves capabilities; the postprocessor produces real machine moves and a motion timeline; sampled analysis then operates on emitted motion. This is substantially better than directly concatenating G-code strings.

The problem is not the existence of approximations. Every practical CAM system contains approximations. The problem is that several approximations are described or certified as guarantees without a proof obligation, residual, or failure state. The most serious example is the gouge certificate, but the same pattern appears in adaptive refinement, constant-scallop naming, work-frame travel checks, entry clearance, and tool-shape approximations.

Finding matrix

ID	Severity	Finding	Primary evidence
CAM-01	CRITICAL	Sampled stock checks are promoted into a gouge-verification certificate although no target surface is compared.	<code>packages/analysis/src/checks.ts:53-121</code> ; <code>packages/compiler/src/recertify.ts:31-59</code>
CAM-02	CRITICAL	Contour endpoint keys collide every 67.108864 mm and can merge unrelated segments.	<code>packages/geometry/src/contours.ts:37-47</code> , <code>140-200</code>
CAM-03	HIGH	Machine travel is checked in a simplified work frame; requested WCS selection is neither transformed nor emitted.	<code>packages/machine/src/profile.ts:33-47</code> ; <code>packages/compiler/src/validate.ts:63-76</code> ; <code>packages/script-host/src/api.ts:217-240</code>
CAM-04	HIGH	Validation promises an appended spindle stop, but the emitter appends only the program-end code.	<code>packages/compiler/src/validate.ts:187-190</code> ; <code>packages/post-rs274/src/index.ts:181-197</code>
CAM-05	HIGH	<code>unsafeCertify</code> is exported and branding is compile-time only, so certificate minting is not a runtime trust boundary.	<code>packages/ir/src/program.ts:70-94</code> ; <code>packages/compiler/src/recertify.ts:20,59</code>
CAM-06	HIGH	Probe emission ignores <code>from</code> and <code>onFailure</code> , and treats travel as an absolute endpoint under <code>G90</code> .	<code>packages/post-rs274/src/index.ts:248-260</code>
CAM-07	HIGH	A missing traverse before a cut is silently repaired with a feed move, potentially through stock or a fixture.	<code>packages/post-rs274/src/index.ts:300-309</code>
CAM-08	HIGH	Raw G-code is not a modal barrier; compression can omit a required motion/feed/axis word after raw text.	<code>packages/post-rs274/src/index.ts:263-265</code> ; <code>packages/compiler/src/gcode-ir.ts:76-142</code>

ID	Severity	Finding	Primary evidence
CAM-09	HIGH	A program's machine ID is not checked against the validating profile.	<code>packages/compiler/src/validate.ts:50-201</code> ; <code>packages/ir</code> program types
CAM-10	HIGH	The postprocessor invents initial position <code>(0,0,clearance)</code> rather than requiring a defined machine-entry state.	<code>packages/post-rs274/src/index.ts:83-84</code>
CAM-11	HIGH	Linking and entry clearance use static stock/part predicates rather than evolving removed stock and swept holder geometry.	<code>packages/planner/src/linker.ts:21-163</code> ; <code>packages/planner/src/entry.ts:87-141</code>
CAM-12	HIGH	Midpoint-only refinement can terminate at depth/length bounds without proving the claimed chord-error guarantee.	<code>packages/planner/src/refine.ts:1-78</code>
CAM-13	HIGH	The experimental constant-scallop strategy does not establish a curvature-aware cusp-height guarantee.	<code>packages/strategies/src/constant-scallop.ts</code> ; <code>packages/geometry/src/eikonal.ts</code>
CAM-14	HIGH	Nearest-neighbor ordering may reverse open paths without climb/conventional or lead-in direction constraints.	<code>packages/planner/src/linker.ts:165-246</code>
CAM-15	HIGH	Operation domains are incompletely validated; zero/negative stepdown, stepover, or angle values can cause invalid paths or guarded loops.	Script API, strategy implementations, planner guards
CAM-16	MEDIUM	Pocket documentation overstates engagement behavior; the first transition can slot diagonally into solid stock.	<code>packages/strategies/src/pocket.ts:84-143</code>
CAM-17	MEDIUM	Marching-squares saddles use a center average rather than the bilinear asymptotic decider.	<code>packages/geometry/src/contours.ts:115-124</code>
CAM-18	MEDIUM	Eikonal sweeping runs a fixed number of rounds with no residual or convergence certificate.	<code>packages/geometry/src/eikonal.ts:72-111</code>
CAM-19	MEDIUM	Several surface strategies operate over padded rectangular fields without a precise machinable-domain mask.	Raster, hybrid, and constant-scallop strategy code
CAM-20	MEDIUM	V-bit and bull-nose contact are approximated without exposing approximation direction or an error bound.	<code>packages/geometry/src/drop-cutter.ts:64-68</code> , <code>302-326</code>
CAM-21	MEDIUM	<code>isFullyVerified</code> can return true while fixture and holder checks are <code>not-checked</code> .	<code>packages/analysis/src/certificate.ts:153-161</code>
CAM-22	MEDIUM	Makera metadata and comments accept unescaped user strings, permitting record/comment line injection.	<code>packages/post-makera/src/index.ts:35-39</code> ; RS-274 formatting paths

ID	Severity	Finding	Primary evidence
CAM-23	MEDIUM	A hard-coded 0.02 mm numerical tolerance is added to certificates without being derived from all participating kernels.	Studio and CLI compile pipelines

CAM-01 - The certificate asserts a gouge check that does not exist

`runSampledChecks` creates a dextral stock simulation, checks low XY rapids, records material removed by rapid moves, and detects motion below the stock bottom. It has no target mesh, target height field, allowance model, protected surface, or deviation comparison. Its result contains `worstRapidCrash`, `worstSpoilboard`, diagnostics, and resolution. There is no gouge result.

`recertify` receives one undifferentiated `SampledEvidence` object and promotes both `gouge` and `rapidCrash` from `not-checked` to `verified-to-resolution`. The Studio and CLI call this path after the same stock-removal simulation. The executable probe started with:

```
{"gouge":{"kind":"not-checked","reason":"probe"}}
```

and returned:

```
{"gouge":{"kind":"verified-to-resolution","spatial":0.5,"numerical":0.02}}
```

No target-surface evidence was supplied.

This is more serious than a missing feature because the public artifact states that the feature ran. A user may reasonably interpret “gouge verified” as evidence that no cut entered protected final geometry beyond tolerance. The current implementation proves only that sampled moves did not produce the specific rapid/spoilboard diagnostics.

Required change. Make evidence type-specific and impossible to cross-apply. For example:

```
interface RapidCrashEvidence {
    kind: "rapid-crash";
    emittedProgramHash: string;
    stockHash: string;
    spatialResolution: Mm;
    diagnostics: readonly Diagnostic[];
}

interface GougeEvidence {
    kind: "target-deviation";
    emittedProgramHash: string;
    targetHash: string;
    stockHash: string;
    toolAssemblyHash: string;
    allowance: Mm;
    spatialResolution: Mm;
    numericalBound: Mm;
    maximumOvercut: Mm;
    unresolvedCells: number;
}
```

Only `GougeEvidence` should be able to mint the gouge row. A real check must compare the simulated remaining stock, swept cutter envelope, or sampled cutter-location surface against a protected target plus allowance. Cells that are not resolved to the advertised bound must produce `inconclusive`, not `verified-to-resolution`.

The certificate should also record the exact emitted-file hash. Otherwise a valid check can be attached to changed output.

CAM-02 - Contour chaining collides inside the machine envelope

Endpoint coordinates are quantized at $1e-6$ mm and then reduced modulo 2^{26} . In coordinate space the key therefore repeats after:

```
67,108,864 * 0.000001 mm = 67.108864 mm
```

The key combines two already-wrapped values, so points separated by that period in X or Y are indistinguishable. This is not a theoretical large-coordinate problem: 67.108864 mm is well inside the Z1's approximately 200 mm XY work envelope.

The runtime regression supplied two disconnected segments whose endpoints differ by exactly that period. The chain result was one polyline with points at `0`, `1`, and `67.108864` mm and `closed: true`.

The resulting failure can propagate into waterlines or iso-scallop paths as a long unintended connector, a false closed loop, or an incorrect inside/outside interpretation. Because the geometry remains syntactically valid, visual inspection at a wide zoom can miss it.

Required change. Do not compress two unbounded quantized coordinates into a bounded numeric key. Suitable alternatives are:

- a nested map `Map<qx, Map<qy, endpoint[]>>`;
- a string or structured tuple key such as ``${qx},${qy}``;
- a collision-free `BigInt` pairing with signed-coordinate encoding;
- preferably, grid-edge topology IDs for marching-squares neighbors, avoiding coordinate matching for segments produced by the same grid.

Add property tests over the complete machine envelope, explicit regressions at positive and negative multiples of 67.108864 mm, and randomized segment sets that compare the optimized chaining result with a slow tuple-key reference.

CAM-03 - Work offsets and machine travel are modeled in incompatible frames

`MachineProfile.travels` explicitly states that physical machine-frame limits are being expressed in the work frame as a simplification. `validate` then compares work coordinates directly with those ranges and calls the result exact. The script API accepts any string as `workOffset`, but the selected offset is not represented as a canonical modal command and is not emitted by the RS-274/Makera postprocessor.

The executable probe requested `G59`; the generated Makera program contained neither `G59` nor an equivalent setup transform. It started with `G90 G21` and proceeded directly to tool and cutting commands.

This means three different things are conflated:

1. the coordinate system used by CAM geometry;
2. the controller's selected WCS register;
3. the transform from that WCS to machine coordinates and physical soft limits.

A path can pass the current travel check while landing outside physical travel for the actual stored offset. Conversely, a valid setup can be rejected because the synthetic work-frame range does not match it.

Required change. Define frames as first-class data. The job must contain a supported WCS enum, an expected setup transform or bounded setup region, and a setup-verification requirement. Lowering should emit WCS selection explicitly. Exact machine-travel verification requires either a known transform or an execution-time controller check against current offsets. Without that evidence the certificate row must be `setup-dependent` or `not-checked`, not exact.

The controller-side deployment manifest should bind the expected WCS and require an operator or probe-derived confirmation immediately before play.

CAM-04 - The promised safety epilogue is not emitted

Validation tracks spindle state. If the program ends with the spindle running, it emits a warning stating that an `M5` will be appended. The RS-274 emitter loops over commands and then appends only `M30` or the configured program-end word. It does not synthesize `M5`.

The runtime probe produced:

```
T1 M6
S5000 M3
M02
```

with no `M5`, despite the validator warning that one would be appended.

Some controllers stop the spindle at program end, but that behavior is dialect- and configuration-dependent. More importantly, the implementation's validation contract is false.

Required change. Normalize a canonical epilogue before validation or emission. At minimum it should express, in target-aware order:

```
retract to verified safe state
M5           spindle off
M9           coolant off
optional safe WCS/machine position
M30 or M02
```

The validator should hard-fail if a target cannot represent the required shutdown semantics. Postprocessor tests should assert the epilogue against every path, including empty programs, raw escapes, exceptions, and target-specific footer hooks.

CAM-05 - Certificate branding is not a trust boundary

The project uses a branded `ValidatedProgram`, which is useful inside TypeScript. However, `unsafeCertify` is exported through the IR package and used by compiler code. At runtime the brand does not exist. Any JavaScript consumer can call the minting function or construct a structurally similar object.

This does not make the compiler architecture invalid, but it means a certificate is evidence only when produced and verified in a controlled pipeline. It cannot be treated as a capability merely because TypeScript types say so.

Required change. Keep certificate construction private to a verification module, emit a serialized certificate with a schema version and hashes, and verify it independently at the deployment boundary. For stronger provenance, sign the manifest in CI or attach a local keyed MAC to a controller-trusted build service. The safety property should be based on verifiable content, not an erased TypeScript brand.

CAM-06 - Probe semantics are emitted incorrectly

The canonical probe carries `from`, a unit direction, maximum travel, feed, and failure behavior. The emitter chooses `G31` or `G38.2` and writes axes as `direction * maxTravel`. The preamble is absolute (`G90`). Therefore a probe beginning at `(100,100,10)` in `Z-` with 5 mm travel becomes:

```
G38.2 X0.000 Y0.000 Z-5.000 F100
```

rather than a move to `(100,100,5)` or a temporary incremental `Z-5`. X and Y are commanded to zero as part of the probe. `onFailure: continue` is also ignored; `G38.2` is the error-on-failure variant on common RS-274 controllers, while no-error variants use different words.

Required change. Define the dialect semantics for probe endpoint and failure mode. Either compute `to = from + direction * travel` in absolute mode or bracket the command with a fully restored incremental-mode sequence. Emit only changing axes. Validate normalized direction, positive travel/feed, start/target travel limits, and target support. Include the emitted probe in the motion timeline and certificate.

CAM-07 - A missing traverse is converted into a cutting move

`emitCut` says that a planner gap should not be hidden, but its condition does exactly that: whenever the current position differs from the cut start by more than 1e-6 mm, it emits a `G1` to the start at cutting feed. The probe generated a feed move from the assumed current point directly to `(10,0,-1)`.

A gap is a compiler invariant violation. Converting it into a feed bridge can cut diagonally through stock, a boss, or a fixture. It also hides the planner defect from validation.

Required change. Reject a non-epsilon gap with an error identifying the previous command and operation. If an explicit transition is wanted, create a canonical traverse or link path before validation and analyze it like every other motion. The postprocessor should never invent material-removing motion.

CAM-08 - Raw text invalidates modal compression

A raw command is emitted as a miscellaneous string, but the modal compressor does not parse it and does not invalidate tracked state. In the probe, a canonical `G1`, raw `G0`, and another intended `G1` compressed to:

```
G1 X1.000 Y0.000 Z0.000 F100.0
G0
X2.000
```

The last line executes under the raw `G0`, not under `G1`. Similar errors can affect feed mode, plane, units, absolute/incremental mode, WCS, spindle state, and coolant state.

The current certificate marks some checks unverifiable after raw text, but leaving feed or other rows exact is still unsafe because raw text may alter any modal group or physical state.

Required change. The safest production policy is to prohibit raw escapes. A development-only escape must either:

- parse into a typed effect set and update every modal state; or
- act as a full modal barrier, causing subsequent output to restate units, distance mode, plane, feed mode, WCS, motion, feed, spindle, coolant, and relevant offsets.

All certificate rows affected by an unparsed escape must become `unverifiable`. Raw strings also need one-line grammar validation to prevent metadata/comment injection.

CAM-09 and CAM-10 - Identity and initial state are assumed

Validation accepts a `MachineProgram` whose `machineId` does not match the supplied profile. The executable probe validated `other-machine` against the Makera Z1 profile. Profile identity should include not just a display ID but a schema/revision/hash, because a changed acceleration, travel, spindle range, rapid semantics, or dialect can invalidate earlier evidence.

Separately, the RS-274 emitter initializes its internal current point to `(0,0,setup.clearance)`. That point is not established by emitted motion or a precondition in the program. Time estimates and the first link are therefore based on an invented location.

Required change. Reject profile ID/revision/hash mismatch before lowering or validation. Make machine-entry state explicit: either require a known start position certified by the setup, emit a safe machine-coordinate initialization sequence, or mark the first position unknown and force an explicit retract/home/setup operation before XY motion.

CAM-11 - Linking and entry use an incomplete clearance model

The linker is a useful separation of concerns, and decomposing non-coordinated rapid intent at the postprocessor is correct. However, clearance decisions use static predicates based on the part/stock top rather than an evolving stock model. A region cleared by an earlier path is not represented; conversely, a path assumed clear at one Z may intersect a fixture, holder, shank, or remaining island.

Helical entry checks 12 XY points at the final target Z. That does not prove the descending helix's swept cutter or holder is clear at every angle and Z. A thin intrusion can fall between samples, and side contact above target depth is not evaluated.

Required change. Maintain an evolving stock representation at operation granularity, and evaluate the swept tool assembly, not only the cutter tip. Link feasibility should have explicit modes:

- verified stay-down through cleared stock;
- verified local lift over remaining stock and fixture envelope;
- full retract to a setup-certified plane;
- unresolved, requiring a diagnostic rather than a guessed move.

Entry should be checked as a continuous swept volume or with an adaptive bound tied to geometry and tolerance. Fixture and holder models must participate before those certificate rows can be called verified.

CAM-12 - Adaptive refinement does not prove its stated bound

`refineSpan` samples only the midpoint. It accepts the span when the midpoint deviation is below tolerance and the span is below `maxSegment`. A narrow feature between an endpoint and midpoint can be missed. The default maximum length reduces that risk but does not mathematically eliminate it. When recursion reaches `maxDepth` or `minSegment`, the function emits the endpoint without reporting that the tolerance may remain unresolved.

The module comments state that the emitted polyline is “guaranteed” to track the surface and that this is what the error-budget contribution claims. The implementation does not establish that guarantee for an arbitrary evaluator.

Required change. Either narrow the contract or strengthen the algorithm. A defensible bound can use known Lipschitz/curvature limits for the evaluator, interval bounds over the triangle/grid cell set, or adaptive multi-point tests with a reported residual. Hitting depth/minimum-segment limits must return an unresolved contribution and block a `verified` claim.

CAM-13 - The constant-scallop name overstates current behavior

The strategy computes a surface-distance field from local slope and a ball-tool flat-surface stepover relation, then extracts contours of an Eikonal arrival field. This is a plausible experimental surface-distance strategy. It is not yet a general constant-cusp-height guarantee because:

- normal curvature in the cross-feed direction is not fully represented;
- the Eikonal field and contour extraction have their own grid errors;
- convergence is not checked;
- the rectangular domain/boundary choice can influence paths;
- cutter-contact and final swept-surface deviation are not measured as the acceptance test.

Required change. Label the current mode `surface-distance` or `experimental-constant-scallop`. Measure the resulting cusp/deviation on the target with the actual cutter model, include field/contour/refinement errors in the budget, and only call it constant-scallop when that measured bound is satisfied.

CAM-14 and CAM-15 - Path semantics and operation domains need stronger contracts

The nearest-neighbor linker can reverse open paths to minimize travel. Reversal changes climb versus conventional milling, lead-in direction, chip evacuation, deflection direction, and possibly the validity of a ramp/entry. Paths need metadata such as `reversible`, `cutSide`, `preferredDirection`, `entryAtStartOnly`, and continuity constraints. The optimizer must respect those semantics before minimizing distance.

The script/API layer does not comprehensively enforce positive, finite, and ordered domains. Strategy loops contain large iteration guards, but a guard that silently truncates output is not input validation. Zero or negative stepdown, invalid stepover, degenerate stock, impossible floor depth, nonpositive feed, or invalid tool geometry should fail before planning.

Required change. Centralize schemas and invariants at the plan boundary. Any iteration guard reached during valid input should be a hard compiler error with operation provenance, not a partial path.

CAM-16 through CAM-23 - Additional algorithm findings

Pocket engagement. The pocket implementation is useful for a prototype, but the claim that it grows from an open center without full-width engagement is not generally true. The first move can diagonally slot into solid stock. Require an explicit cleared entry, helix, drill, or trochoidal/engagement-aware first ring.

Saddle topology. A center average is simple but is not the full bilinear asymptotic decider for ambiguous marching-squares cases. Use the determinant/bilinear interpolant at the saddle and add topology tests near equality.

Eikonal convergence. Four fixed sweep rounds may be enough for many grids but offer no convergence evidence. Iterate until a residual threshold or a bounded maximum; report non-convergence.

Domain masks. Padded rectangular grids can generate paths outside the intended machinable region or across holes/occlusions. Carry an explicit domain mask, boundary conditions, and inaccessible-cell state through field construction and contour extraction.

Tool geometry. The V-bit model behaves as a flat tip disc for contact, and the bull-nose model combines simpler surfaces without a stated conservative direction or error. Expose exact/approximate status and bound. Prohibit approximate tools from final verification when the bound exceeds the job tolerance.

Certificate completeness. `isFullyVerified` ignores fixture and holder rows. The runtime probe returned `true` while both were `not-checked`. Rename the predicate to the exact subset it tests or require every safety-relevant row, including setup, fixture, holder, tool assembly, and profile identity.

Text injection. Makera metadata fields use delimiter-separated records, and comments are emitted from user-controlled descriptions. Reject CR/LF/NUL and reserved delimiters, or use a length-prefixed/escaped representation. Structured metadata should be generated only from normalized values.

Numerical tolerance. Both Studio and CLI recertification use a fixed 0.02 mm numerical tolerance. A certificate should derive contributions from mesh quantization, acceleration structure, contact model, field grid, contour interpolation, chord refinement, decimal formatting, arc linearization, dixel resolution, and machine/controller resolution. A constant untraced number is not an error budget.

JavaScript API and CAM language design

Summary

The capability-shaped API is a productive authoring model. It is substantially easier to review than scripts that emit arbitrary G-code, and the runtime unit tags are a good response to JavaScript's lack of static units. A fresh API object is created per run, operations are harvested into a declarative `ManufacturingPlan`, and diagnostics retain enough context for an interactive editor.

The current implementation should not, however, be described as a secure sandbox or a fully specified CAM language. It is unrestricted imperative JavaScript executed in the host realm, with a capability object layered on top. The language's semantics also depend on mutable JavaScript objects, synchronous callbacks, ambient globals, and several undocumented defaults. Those are manageable for an experimental personal tool, but not for a machine-control authoring boundary.

Finding matrix

ID	Severity	Finding	Primary evidence
JS-01	CRITICAL	Script execution uses same-realm <code>new Function</code> ; prototype-chain escape reaches host globals and <code>timeoutMs</code> is not enforced.	<code>packages/script-host/src/sandbox.ts:1-20</code> , <code>47-99</code> ; Studio compile thunk
JS-02	HIGH	Boxed units bypass finite checks, and domain invariants are not centralized.	<code>packages/script-host/src/api.ts:91-106</code> , <code>127-169</code>
JS-03	HIGH	Tool objects are mutable references stored directly in the plan.	<code>packages/script-host/src/api.ts:138-143</code>
JS-04	HIGH	Scope combinators are synchronous but accept async functions at runtime, restoring state before awaited work.	<code>packages/script-host/src/api.ts:250-260</code>
JS-05	HIGH	<code>withSpindle</code> does not emit balanced state changes, and speed changes for the same tool are lost.	API comments; <code>packages/planner/src/run.ts:82-99</code>
JS-06	HIGH	<code>workOffset</code> is an arbitrary string and has no execution semantics.	<code>packages/script-host/src/api.ts:217-240</code> ; postprocessors
JS-07	MEDIUM	Ambient nondeterminism and host-version behavior are not removed or included in artifact identity.	Script host and JavaScript globals
JS-08	MEDIUM	Full imperative JavaScript is the authoring language and audit surface; there is no versioned pure plan schema as the stable language contract.	Script host/API/planner boundary
JS-09	MEDIUM	Handbook and implementation drift on worker isolation, spindle scopes, accepted invalid values, and line mapping.	<code>ApiHandbook.tsx</code> ; script host implementation

JS-01 - This is not a security boundary and has no timeout

The module comment correctly admits that shadowing globals is not a security boundary. The implementation still compiles user text with `new Function`, passes API values as parameters, and calls the result in the current realm. Shadowing `globalThis`, `process`, `fetch`, and similar names prevents accidental direct access, but standard constructors and prototypes remain reachable.

The executable probe used a constructor-chain path from an exposed function and reached the host global object. In Node it read `process.version` as `v22.16.0`. This demonstrates that the capability list is not the effective authority boundary.

The `RunOptions.timeoutMs` comment says enforcement belongs to a worker caller, but `runScript` does not inspect the value. The Studio compile thunk calls `runScript` directly in its async Redux thunk, on the browser main thread. A probe requested a 1 ms timeout, ran a busy script for approximately 182.5 ms, and returned successfully. An infinite loop would freeze the tab and cannot be interrupted by an `AbortSignal` checked only later in planning.

The consequences are broader than availability. A shared or generated script can potentially access DOM/storage/network in a browser through escaped globals, read CLI environment and files in Node, mutate built-ins for later compilations, or exfiltrate project data.

Required change. Use a hard isolation boundary:

- In the browser, create a dedicated Worker for each compilation or a short-lived worker pool. Terminate the worker on wall-clock timeout. Apply a restrictive Content Security Policy and do not grant network/storage capabilities.
- For stronger language isolation, run code in SES/Compartment with frozen intrinsics, or in a QuickJS/WASM interpreter whose host calls are explicitly marshalled.
- In the CLI, use a subprocess or isolated runtime with CPU/wall-clock and memory limits, a minimal environment, no inherited file descriptors, and no filesystem/network access unless explicitly granted.
- Serialize the plan across the boundary and validate it again in the trusted compiler process. Never trust object identity or prototypes from the script realm.

Add adversarial tests for constructor escape, async constructor, prototype pollution, dynamic import, WebAssembly, shared-memory allocation, memory exhaustion, event-loop starvation, and post-timeout cleanup. A timeout test must prove that execution is terminated, not merely that the caller stopped waiting.

JS-02 - Unit tags do not enforce numeric validity or machining domains

Bare numbers are checked with `Number.isFinite`. Boxed unit values are only checked for their `__unit` tag and then return `b.v` directly. The constructors themselves box any JavaScript number. The probe therefore accepted `mm(Infinity)` as a tool diameter and produced a successful plan containing an infinite value.

Even finite numbers need domain checks. Examples include:

- tool diameter, tip diameter, corner radius, flute length, feed, spindle speed, stepdown, stock dimensions, and clearance must be positive;
- corner radius must fit the tool radius;
- V-bit tip diameter must not exceed major diameter and included angle must be in a valid open interval;

- stepover ratios need a defined interval, normally `0 < stepover <= 1` unless a strategy explicitly supports another range;
- floor depth and operation depth must agree with stock/setup conventions;
- boxes and meshes must have finite, ordered bounds;
- direction vectors must be finite and normalized where required.

Scattered checks in later algorithms make failure timing unpredictable and can permit nontermination or partial results.

Required change. Parse the harvested plan through one trusted, versioned runtime schema. Use branded constructors only as authoring conveniences; the schema remains authoritative. Diagnostics should name the operation, field, supplied value, expected range, and unit. Reject `NaN`, infinities, negative zero where semantically relevant, accessors, proxies, symbols, unexpected keys, and cyclic objects at the isolation boundary.

JS-03 - Tool definitions are mutable after registration

`defineTool` creates a normal object, stores the same reference in the tool table, and returns it. A script can mutate its name, number, geometry, holder, or other fields after operations have referenced its ID. The runtime probe changed a tool to name `after`, number `99`, and diameter `123`; those mutations appeared in the harvested plan while the operation retained tool ID `T1`.

This breaks assumptions about stable identity and can create disagreement between operation references, T-numbers, header metadata, simulation lookup, and the object seen by later code.

Required change. Return an opaque tool handle containing only an immutable ID. Deep-clone and deep-freeze the normalized tool record inside the host, or copy it into a realm-neutral serialized plan immediately. Number assignment should be validated for uniqueness and target range. The compiler should hash normalized tool and assembly definitions and include those hashes in evidence.

JS-04 - Async callbacks violate lexical scope semantics

`withTool` and `withSpindle` call `body()` inside synchronous `try/finally`. JavaScript permits an `async` function to be passed despite the TypeScript callback type when scripts are plain JavaScript. The function returns a Promise immediately; the `finally` restores state before awaited statements continue.

The probe produced no synchronous motion, returned a `program.noMotion` diagnostic, and later generated an unhandled rejection because the resumed async body no longer had an active tool.

Required change. Choose one explicit semantic model:

- For a synchronous language, detect thenables returned by scope callbacks and fail immediately with a specific diagnostic before harvesting. Also capture unhandled rejections inside the isolated worker.
- For an asynchronous language, make `runScript` `async`, await each scope body, preserve state across awaits, and terminate outstanding work at the compile deadline.

The synchronous model is preferable for deterministic CAM authoring unless there is a concrete need for asynchronous capabilities.

JS-05 - Spindle scope documentation and execution disagree

The API comment states that `withSpindle` emits start, executes the body, and emits stop so the user cannot express an unbalanced spindle. The implementation only annotates each operation with the current speed. The planner starts the spindle if it is not running, generally at the first operation or after a tool change. It does not compare the requested speed of subsequent operations using the same tool.

The runtime probe created two operations at 5000 and 9000 RPM. The canonical output contained a start at 5000 RPM and a final stop; it did not contain a transition to 9000 RPM. An operation outside a spindle scope receives the planner's silent 12000 RPM default.

This is a language-semantics defect: the program a user reads is not the program executed.

Required change. Represent spindle transitions explicitly in the plan or define exact operation-level semantics. Before every operation, the runner should compare desired mode/speed with current state and emit a transition when needed. Exiting a lexical scope should emit the prior state or off state as documented. Prefer requiring an explicit spindle policy rather than defaulting to 12000 RPM. Tests should cover nested scopes, same-tool speed changes, exceptions, empty scopes, tool changes, and operations outside a scope.

JS-06 - `workOffset` is a dead and unsafe language feature

The API accepts `workOffset?: string`, defaults it to `G54`, and stores it. Any string is accepted. The field is not emitted, does not drive a transform, and is not consumed by controller deployment.

A language feature that appears to select machine setup but has no effect is worse than an omitted feature because it creates false confidence.

Required change. Replace the string with a closed enum supported by the selected machine profile. Make it a required setup datum or provide an explicit default shown in generated artifacts. Lower it to a canonical coordinate-system selection, emit it, verify it in post round-trip tests, and bind it into the deployment manifest/operator check.

JS-07 - Reproducibility is not a language invariant

The host shadows some dangerous names but does not create a deterministic standard library. `Math.random`, `Date`, locale behavior, floating-point implementation details, object enumeration from user-created data, and host runtime/version can influence a script. Constructor escape also restores all ambient authority.

For CAM, reproducibility matters because a certificate must correspond to the exact output. The same source and assets should either produce the same normalized plan and G-code or explicitly record all nondeterministic inputs.

Required change. Provide a deterministic realm with frozen intrinsics, a seeded PRNG capability if randomness is genuinely needed, stable sorting, normalized numeric formatting, and no current-time API. Hash source, imported assets, language/compiler versions, machine profile, tool library, options, and seed. Run reproducibility tests across browser worker and CLI runtime.

JS-08 - Separate the authoring frontend from the stable CAM language

Full JavaScript is convenient for loops, functions, and parameterization, but it is a poor long-term interchange and audit format. Static analysis cannot reliably determine bounds, tool usage, side effects, termination, or which branches contributed to the plan. A reviewer must execute arbitrary code to know the job.

The stable language should be the normalized declarative plan, not the script. JavaScript can remain one frontend that builds the plan.

A target shape is:

```
const job = cam.job({
  schema: "dropcut.plan/v1",
  machine: { id: "makera-z1", profileRevision: "..." },
  setup: {
    stock: cam.box({ x: mm(100), y: mm(80), z: mm(12) }),
    wcs: "G54",
    clearance: mm(8),
  },
});

const finish = job.tool("finish", cam.ballEndMill({
  diameter: mm(3),
  fluteLength: mm(12),
}));

job.operation({
  id: "finish-top",
  tool: finish,
  spindle: { mode: "cw", speed: rpm(12000) },
  feed: mmPerMin(350),
  geometry: { mesh: "part", allowance: mm(0) },
  strategy: cam.surfaceDistance({ cuspTarget: mm(0.03) }),
  entry: cam.autoEntry({ maxRampAngle: deg(3) }),
  direction: { milling: "climb", reversible: false },
});

export default job.finish();
```

After execution in the untrusted frontend, the result should be plain versioned data with no functions, prototypes, getters, or shared mutable references. A trusted parser validates and canonicalizes it. The Studio should display this normalized plan and its hash alongside source, so reviewers can inspect what the script actually declared.

JS-09 - The handbook is ahead of the implementation

The handbook says the compiler catches non-finite values and that spindle scopes restore state; module comments say browser execution occurs in a module worker and that scope combinators make unbalanced spindle state impossible. The current Studio uses direct same-thread execution, boxed infinities pass, and spindle speed restoration is not emitted.

Documentation is part of a language contract. Executable examples should compile in tests, and claims should be generated from supported features rather than manually duplicated prose.

Required change. Add handbook conformance tests that extract or share example source, compile it, and compare normalized plan plus G-code snapshots. Expose language/compiler version in the UI and artifact. Use source maps or a parser wrapper for reliable line/column diagnostics; line offsets inferred from generated-function stack text are best-effort and engine-dependent.

Cross-system trust, deployment, and operations

The missing boundary is a deployment bundle

The CAM compiler and controller currently meet only at a filename. `z1ctl job run` accepts any local file, uploads it, verifies transfer integrity using the firmware's MD5/size mechanism, performs current machine-state preflight, sends `play`, and monitors status. This is a good transfer workflow, but it has no knowledge of CAM provenance or evidence.

The transfer digest answers: "Did these bytes reach the SD card?" It does not answer:

- Were these bytes produced by this compiler?
- Do they correspond to the certificate being shown to the operator?
- Were they compiled for this machine profile and revision?
- Which WCS, stock, fixture, holder, and tools were assumed?
- Has the file been edited after simulation?
- Did every required verification stage run and pass?

A production path should deploy a bundle, not a loose `.nc` file. One possible manifest is:

```

{
  "schema": "dropcut.deployment/v1",
  "artifact": {
    "file": "part.nc",
    "sha256": "...",
    "bytes": 123456
  },
  "source": {
    "scriptSha256": "...",
    "normalizedPlanSha256": "...",
    "assets": [{"name": "part.stl", "sha256": "..."}]
  },
  "compiler": {
    "version": "...",
    "commit": "...",
    "postprocessor": "makera/v1"
  },
  "machine": {
    "id": "makera-z1",
    "profileRevision": "...",
    "profileSha256": "...",
    "firmwareRange": "..."
  },
  "setup": {
    "wcs": "G54",
    "stockSha256": "...",
    "fixtureSha256": "...",
    "toolAssemblySha256": "...",
    "requiredAttestations": ["homed", "wcs-probed", "tool-loaded"]
  },
  "certificate": {
    "sha256": "...",
    "status": "conditional",
    "blockingRows": []
  }
}

```

`z1ctl` should parse this manifest, verify the SHA-256 of the exact G-code, compare machine/profile/firmware identity, display unresolved certificate rows, and require setup attestations close to play time. The firmware-required MD5 can remain as transport integrity, but SHA-256 should bind the local bundle.

SYS-01 - Job start does not prove the job actually started

The controller sends `play`, then begins polling. If playback is active it reports progress. Otherwise it immediately reports completion. The pinned stock firmware evidence documents that `play` silently returns when the machine is not homed. The current preflight cannot reliably prove homing and treats that row as advisory.

Therefore “never started” and “finished” can follow the same success path.

Required change. Add a start phase with a deadline and an observed transition into a playing/active state. Record the initial status, send play, then require a distinct active state or target-file identity before monitoring completion. A machine that cannot report this reliably needs a separate operator attestation and an explicit `start-unverified` result, not success.

SYS-02 - Controller session state is not a safety state machine

The control library has sound local checks but lacks one authoritative session model spanning connection, protocol negotiation, alarms, homing confidence, transfer mode, preflight, operation ownership, hold/resume, timeout quarantine, and reconnect. As a result, behavior is distributed across mutexes, channels, helper functions, and caller conventions.

A more robust model would make illegal transitions unrepresentable:

```
Disconnected
-> Handshaking
-> Ready(read-only)
-> Admitted(operation token)
-> Executing
-> Held
-> Completing
-> Ready

Any ambiguous timeout, decoder loss, protocol switch, or short write
-> Quarantined
-> close/reconnect/handshake
```

Stop and emergency paths remain out-of-band and always admissible. Cycle start, resume, play, jog keepalive, and filesystem mutation require the correct state/token. Safety events are latched independently of command replies.

SYS-03 - The web UI is locally hardened but remote mode needs a secure transport story

The web server's loopback default and request protections are good. Remote mode, however, serves HTTP and presents a bearer token in a URL. Query tokens leak through browser history, logs, copied URLs, screenshots, and potentially referrers. Plaintext traffic exposes both token and commands to an observer on the network.

Required change. Keep direct serving loopback-only for production. For remote access, require a documented TLS reverse proxy, mTLS, SSH tunnel, WireGuard/Tailscale-style trusted network, or equivalent. Put short-lived credentials in an `Authorization` header or secure same-site cookie, not the URL. Add explicit origin allowlisting, token rotation, audit logging, and rate limits. The web process must not run with broader filesystem/network privileges than required.

Reproducibility and test status

The supplied archive is not independently reproducible as delivered:

- `.git` points to an absolute worktree path on the author's machine.
- The root declares `pnpm 10.15.1`, but dependencies were not vendored and outbound package fetch was unavailable in the review environment.
- The Go module requires Go 1.26.1, while the available toolchain was 1.23.2.
- `go.mod` replaces `github.com/go-go-golems/glazed` with an absent `../../glazed` checkout.
- The archive contains no `go.sum` or `go.work`, although the README says the workspace supplies one.

- The nested README says motion and job control are not implemented, despite the reviewed implementation.

The review still completed a dependency-free core TypeScript typecheck and compiled/imported core packages for executable probes. Go source was formatted cleanly, but the full Go suite could not be built in this environment. The repository contains 16 TypeScript test files with approximately 249 `it / test` cases and 8 Go test files with 87 top-level tests. Those counts show substantial testing effort, but test presence is not a substitute for a reproducible clean-room run.

Required change. Add a hermetic CI/release path that produces:

- a portable Git checkout or source tarball with commit metadata;
- an immutable lockfile and cached/package-verified dependencies;
- a supported, released Go toolchain declaration;
- `go.sum` and either a committed `go.work` or no local replace;
- a pinned/tagged `glazed` dependency or vendored module;
- an SBOM and license inventory;
- test, fuzz, static-analysis, and artifact-hash results attached to the release.

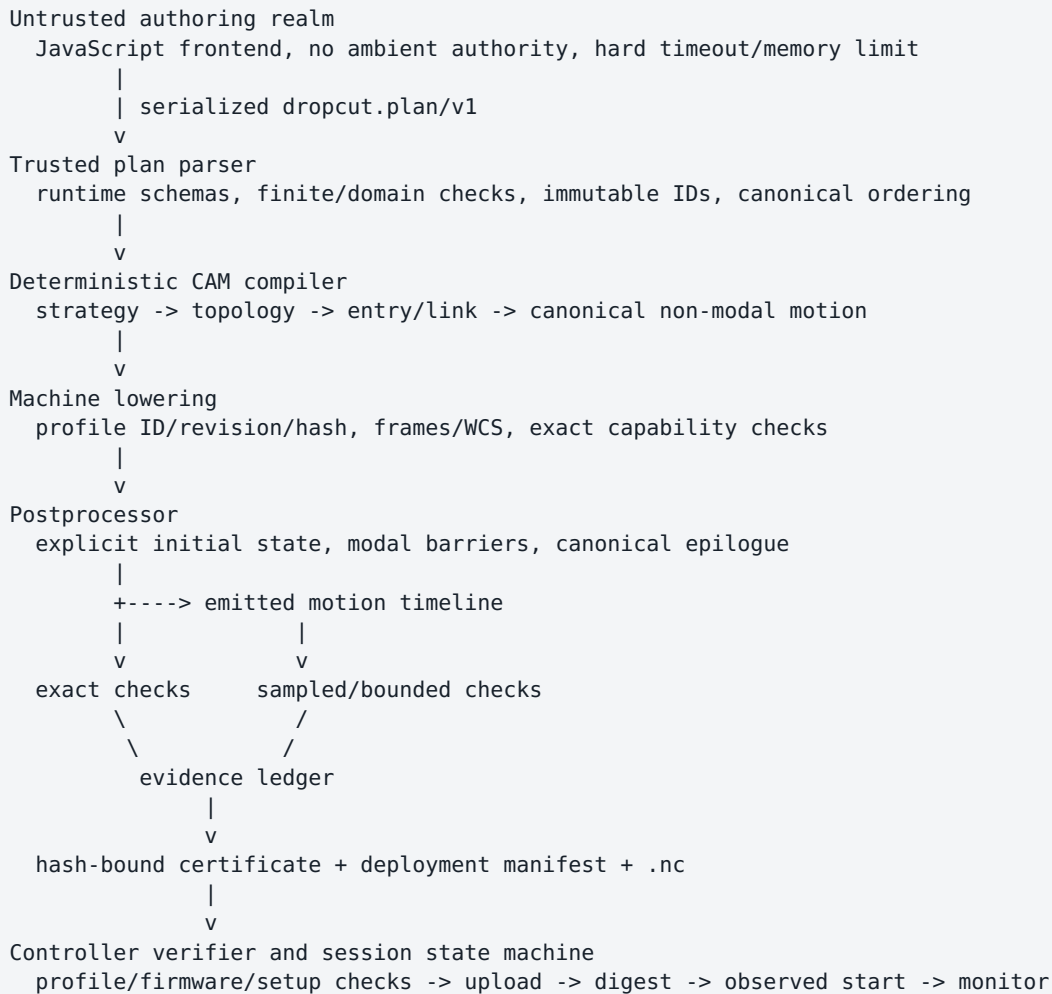
Licensing and process boundaries

The root CAM project is MIT licensed. The nested `makera-z1-cli` is GPL-2.0-only and explicitly states that code linking `pkg/makera` must be GPL, recommending a subprocess/JSON boundary for permissive or proprietary interfaces. That architectural boundary is sensible and should be made operationally explicit.

This report does not provide legal advice. From an engineering packaging perspective, keep the controller as a separate executable/package, communicate through a versioned local protocol, retain notices and source-offer obligations, and generate an SBOM that distinguishes the MIT CAM frontend, GPL controller, and vendored GPL firmware excerpts. Do not accidentally bundle/link the Go controller into a permissive application without license review.

Recommended target architecture

A safer end state preserves the project's semantic-CAM direction while tightening each trust boundary:



The evidence ledger should be append-only in the logical sense: each check records its inputs, algorithm/version, assumptions, resolution/bounds, result, and artifact hash. Recertification should not mutate an undifferentiated status object; it should derive a new certificate from specific evidence records.

Remediation plan

P0 - Required before cutting material

1. **Correct certificate semantics.** Remove the false gouge promotion immediately, even before a true gouge checker exists. Split check-specific evidence and hash it to emitted output.
2. **Isolate scripts.** Move Studio execution into a terminable worker/isolate and CLI execution into a constrained subprocess/runtime. Enforce wall-clock and memory limits.
3. **Make command authorization fail closed.** Remove generic free-text commands from ordinary clients or parse one strictly allowed read-only grammar across the entire payload.
4. **Fix contour keys.** Replace modulo-packed endpoint keys and add work-envelope/property regressions.
5. **Implement frames/WCS/profile identity.** Validate and emit WCS selection; stop claiming exact travel without a machine-frame transform or execution-time evidence.
6. **Fix postprocessor safety semantics.** Canonical M5 / M9 epilogue, explicit initial state, correct probing, hard failure on missing traverses, full modal reset after raw or production ban on raw.
7. **Validate the language boundary.** Reject non-finite and out-of-domain values, deep-copy/freeze definitions, reject async scopes or support them correctly, and emit spindle transitions faithfully.
8. **Bind planning to execution.** Produce and consume a deployment manifest containing G-code hash, machine/profile identity, setup, tools, and certificate.

A limited material trial should not begin until all eight have executable regression tests and a reviewable closure record.

P1 - Required before production or unattended use

- Add an operation-wide controller admission token and priority stop lane.
- Quarantine/reconnect after timeout, partial write, decoder loss, or protocol anomaly.
- Separate command replies, telemetry, latched alarms, and lossy logs.
- Require an observed job-start transition and distinguish unknown/never-started/completed states.
- Add evolving stock, fixture, shank, and holder models to entry/link/crash checks.
- Add path direction/engagement semantics and preserve them through optimization.
- Replace heuristic numerical claims with bounded or measured residuals.
- Establish a secure remote-access deployment rather than direct plaintext HTTP.
- Make release builds hermetic and attach provenance, SBOM, test/fuzz results, and signed hashes.

P2 - Quality and capability maturation

- Upgrade surface-distance finishing to measured constant-cusp behavior.
- Add exact or bounded V-bit/bull-nose/tool-assembly contact models.
- Add domain masks, robust saddle topology, and Eikonal convergence diagnostics.
- Add acceleration/jerk-aware timing and machine-dynamics constraints.
- Expand setup probing and machine-frame verification.
- Generate handbook/reference material from versioned schemas and executable examples.

- Add a migration/version policy for scripts, normalized plans, profiles, manifests, and certificates.

Acceptance gates

Gate A - Script host

A release candidate passes only when:

- constructor/prototype escape cannot access host global authority;
- a busy loop is forcibly terminated near the configured deadline;
- memory exhaustion is bounded and the worker/process is disposable;
- no DOM, storage, network, environment, or filesystem capability exists unless explicitly granted;
- async callbacks have defined semantics and no unhandled work survives harvest;
- identical source/assets/options produce identical normalized-plan hashes across supported runtimes.

Gate B - Language and plan validation

- Every scalar is finite and within a documented domain.
- Tools, stock, setup, operations, and strategy objects are immutable normalized data.
- IDs and tool numbers are unique and stable.
- Work offsets are a closed target-supported enum.
- Invalid loops cannot reach an iteration guard; reaching a guard is a hard diagnostic.
- Handbook examples are compiled as tests.

Gate C - Geometry and algorithms

- Contour chaining matches a collision-free reference over randomized work-envelope data.
- Regressions cover +/-67.108864 mm periods and grid-boundary/saddle cases.
- Entry and linking are checked against evolving stock plus fixture/tool assembly.
- Path reversal obeys directional metadata.
- Refinement and Eikonal stages report residual/convergence and fail unresolved bounds.
- Tool-model approximations declare direction and maximum error.
- Claimed scallop/gouge tolerances are measured on the final swept result.

Gate D - Compiler and postprocessor

- Machine profile ID, revision, and hash must match at validation.
- WCS selection is represented, emitted, parsed back, and bound to setup.
- Initial position/preconditions are explicit.
- Every successful program has a target-correct spindle/coolant/retract epilogue.
- Probe endpoint and failure behavior round-trip correctly.
- Raw text is forbidden or forces a complete modal barrier and certificate downgrade.

- A missing traverse is a compiler failure.
- `parse(emit(P))` preserves motion and state for compressed and uncompressed output, including adversarial raw/modal cases.

Gate E - Certificate

- Every certificate row has a corresponding typed evidence record.
- Evidence records include input and emitted-artifact hashes.
- Gouge evidence contains a target/allowance comparison, not merely stock removal.
- Fixture, holder, setup, tool assembly, and profile identity are included in completeness predicates.
- Hitting numerical/iteration/resolution limits produces `inconclusive`, never pass.
- The controller independently verifies the serialized certificate/manifest schema and hashes.

Gate F - Controller

- Unknown, compound, multiline, malformed, and future commands are denied on read-only paths.
- Fuzz/property tests prove the classifier never understates the maximum risk of a payload.
- Preflight through final command is one admitted operation; stop remains out-of-band.
- Partial writes, delayed sentinels, duplicate/lost messages, alarm floods, and protocol-announcement injection are fault-injected in tests.
- Any ambiguous exchange enters quarantine and requires reconnect/resynchronization.
- Alarms and cover/endstop safety events are latched and cannot be drained as chatter.
- Job completion requires a previously observed job-start transition.

Gate G - Physical bring-up

After software gates pass, use staged hardware validation:

1. fake transport and recorded firmware sessions;
2. connected machine with motors/spindle disabled where possible;
3. homing/setup/probe verification without a tool;
4. physically attended air cuts above stock;
5. sacrificial foam or machinable wax at conservative parameters;
6. soft material with independent G-code and setup review;
7. production material only after repeatability and fault-recovery evidence.

At every powered stage, the operator must have immediate access to a hardware emergency stop. Software stop paths are not a substitute for machine-level protective functions.

Overall conclusion

The implementation has the beginnings of a strong semantic CAM and machine-supervision system. Its best choices should be retained: non-modal canonical motion, capability-driven lowering, emitted-motion analysis, typed control operations, explicit risk classes, no motion retry after ambiguous failure, fresh preflight, and dead-man jog behavior.

The current safety story is nevertheless stronger than the implementation. The report's four critical findings are independent release blockers:

- the certificate can claim an unperformed gouge check;
- scripts can escape the advertised capability boundary and cannot be timed out;
- the controller's generic command path can under-classify dangerous payloads;
- contour topology can be corrupted by an in-envelope key collision.

Several high-severity issues would remain after those four: machine/work frames and WCS are disconnected, postprocessor shutdown/probe/modal behavior is inconsistent, compiler invariants are silently repaired with cutting motion, language values are mutable or insufficiently validated, and execution is not bound to planning evidence.

Accordingly, the correct present label is **research prototype**. Use it for simulation, algorithm development, and supervised air-cut validation. Do not use it as the sole authority for production cutting, unattended jobs, or remote machine operation until the P0/P1 gates are closed and physical validation demonstrates the complete system under expected faults.

Appendix A - Executable probe results

The focused probes were run against compiled core TypeScript modules from the supplied snapshot. They are small counterexamples, not exhaustive testing.

Probe	Requested/constructed state	Observed result	Finding
Sandbox escape	Script receives only capability API names	Constructor chain reached host global and read Node <code>v22.16.0</code>	JS-01
Timeout	<code>timeoutMs: 1</code> with busy computation	Completed successfully after about 182.5 ms	JS-01
Boxed infinity	Tool diameter <code>mm(Infinity)</code>	Plan accepted; diameter remained infinite	JS-02
Mutable tool	Mutate returned tool after definition	Harvested name/number/diameter changed to <code>after / 99 / 123</code>	JS-03
Async tool scope	<code>withTool(tool, async () => ...)</code>	Scope restored before continuation; no motion plus unhandled rejection	JS-04
Spindle scope	Same tool, operations at 5000 and 9000 RPM	Emitted only 5000 RPM start and final off	JS-05
Work offset	Setup requests <code>G59</code>	Output contains no <code>G59</code>	CAM-03 / JS-06
Safety footer	Program ends with spindle running	Validator says M5 appended; output ends <code>S5000 M3 , M02</code>	CAM-04
Probe	From <code>(100,100,10)</code> , Z-, travel 5, continue	Emits <code>G38.2 X0 Y0 Z-5</code> under <code>G90</code>	CAM-06
Missing traverse	Current point differs from cut start	Emitter inserts direct <code>G1</code> to cut start	CAM-07
Raw modal state	<code>G1</code> , raw <code>G0</code> , intended <code>G1</code>	Compression emits final bare <code>X2</code> , which remains rapid	CAM-08
Gouge certificate	Gouge initially <code>not-checked</code> ; generic sampled evidence	Gouge becomes <code>verified-to-resolution</code>	CAM-01
Machine identity	Program machine ID <code>other-machine</code> , Z1 profile	Validation succeeds	CAM-09
Certificate completeness	Fixture and holder <code>not-checked</code>	<code>isFullyVerified</code> returns true	CAM-21
Contour key	Segments separated by 67.108864 mm	Merged into one chain and marked closed	CAM-02

Appendix B - Build, test, and provenance record

Item	Result
Supplied archive	/mnt/data/dropcut-studio.zip
Archive SHA-256	61de82451c552eddcac08dc5f192b81c2db5611734747b571ad4fd69a02be758
Public repository	wesen/dropcut-studio
Review branch	task/cnc-control-dropcut
Review anchor	e82bed1e5a00f38f4441e6ea13e1265edc775928
Stock firmware evidence	MakeraInc/ CarveraFirmware@1683b6fb5c7ec1d341c476c6fdb2a22f7a26220e
Community firmware evidence	Carvera-Community/ Carvera_Community_Firmware@9ac0123018a3f221f18db35cc11667203cd332bd
TypeScript/TSX inventory	87 files; approximately 15,174 lines
Go inventory	50 files; approximately 11,973 lines
Markdown design/evidence inventory	67 files; approximately 22,425 lines
TypeScript test inventory	16 files; approximately 249 test cases
Go test inventory	8 files; 87 top-level tests
Core TypeScript typecheck	Passed for 49 dependency-free non-test core files
Core executable probes	Completed; results summarized in Appendix A
Full pnpm/Vitest suite	Not run; dependencies unavailable offline
Go format	Clean
Full Go suite	Not run; Go version and absent local glazed workspace blocked build
Physical-machine validation by reviewer	Not performed

Review-environment commands and failure modes

The following limitations should be considered when reproducing this audit:

```
Node available:      v22.16.0
TypeScript available: 5.8.3
Go available:        1.23.2
Go required by module: 1.26.1
Package manager:     pnpm unavailable; Corepack fetch blocked by DNS
Go workspace:        ../../glazed absent; go.work and go.sum absent
Upstream git clone:   outbound Git DNS unavailable
```

These limitations prevented a clean-room execution of the full repository test suites. They do not invalidate the source-local findings or the executable core probes, but they reduce confidence in untouched areas. Closing the reproducibility defects is therefore part of the release work, not an administrative afterthought.

Appendix C - Severity model

Severity	Meaning in this review
CRITICAL	A direct path to unsafe machine behavior, false safety evidence, loss of the authoring isolation boundary, or silent geometry corruption under normal operating ranges. Blocks all material cutting.
HIGH	A plausible unsafe or materially incorrect result requiring ordinary conditions, or a systemic recovery/state defect. Blocks production and generally blocks material trials until controlled.
MEDIUM	A defense-in-depth, numerical-honesty, operational-security, documentation, or reproducibility defect that can amplify other failures. Must be scheduled before production.
LOW	Maintainability or usability issue with limited direct safety impact. Not separately enumerated where it did not alter release disposition.

Severity reflects consequence and reachability in this implementation, not a formal ISO/IEC risk score. Physical machine hazards require a separate machinery risk assessment.

Appendix D - Review checklist by subsystem

Machine control

- ☒ Transport framing and protocol selection inspected
- ☒ Command/reply correlation and timeout behavior inspected
- ☒ Risk classification and raw command paths inspected
- ☒ Motion admission, preflight, stop/resume, and jog paths inspected
- ☒ File upload, digest verification, play, and monitoring inspected
- ☒ Browser control origin/token/loopback behavior inspected
- ☒ Stock firmware command behavior checked against pinned excerpts
- ☐ Hardware fault injection performed
- ☐ Physical Z1 motion/spindle behavior independently reproduced

CAM and postprocessing

- ☒ Plan, strategies, planner, lowering, validation, and emission traced end to end
- ☒ Drop-cutter, contour, field, refinement, entry, and linking assumptions inspected
- ☒ Modal compression and round-trip risk inspected
- ☒ Work frame, WCS, profile identity, and travel checks inspected
- ☒ Probe, raw escape, initial state, and epilogue behavior probed
- ☒ Sampled analysis and certificate re-issuance inspected and probed
- ☐ Industrial geometry corpus/differential CAM comparison run
- ☐ Physical swept-volume verification performed

JavaScript language

- ☒ Capability surface and fresh-run state inspected
- ☒ Escape and timeout behavior probed
- ☒

- ☒ Runtime units and invalid numeric values probed
- ☒ Tool mutability and callback scope semantics probed
- ☒ Spindle and work-offset language semantics probed
- ☒ Handbook claims compared with implementation
- ☐ Hardened isolated-runtime implementation reviewed
- ☐ Cross-runtime deterministic corpus verified

Appendix E - Minimum release checklist

- ☐ Remove false gouge verification and invalidate existing affected certificates
- ☐ Isolate and forcibly terminate user scripts
- ☐ Deny unknown/compound/multiline generic commands
- ☐ Replace contour endpoint key packing
- ☐ Validate and emit WCS; bind machine profile revision/hash
- ☐ Add explicit safe initial state and canonical shutdown epilogue
- ☐ Correct probe endpoint/failure semantics
- ☐ Fail on planner-to-cut gaps
- ☐ Ban raw G-code in production or implement a complete modal barrier
- ☐ Enforce finite/domain-valid immutable plan data
- ☐ Correct async scope and spindle-transition semantics
- ☐ Bind `.nc`, certificate, setup, and profile in a controller-consumed manifest
- ☐ Quarantine controller sessions after ambiguous transport/protocol failures
- ☐ Require observed job start before reporting completion
- ☐ Make both TypeScript and Go builds reproducible from a clean checkout
- ☐ Complete staged physical validation with recorded evidence